

Sequence Processing Transformers

Dr. Saeedeh Momtazi



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)



آکادمی هراه

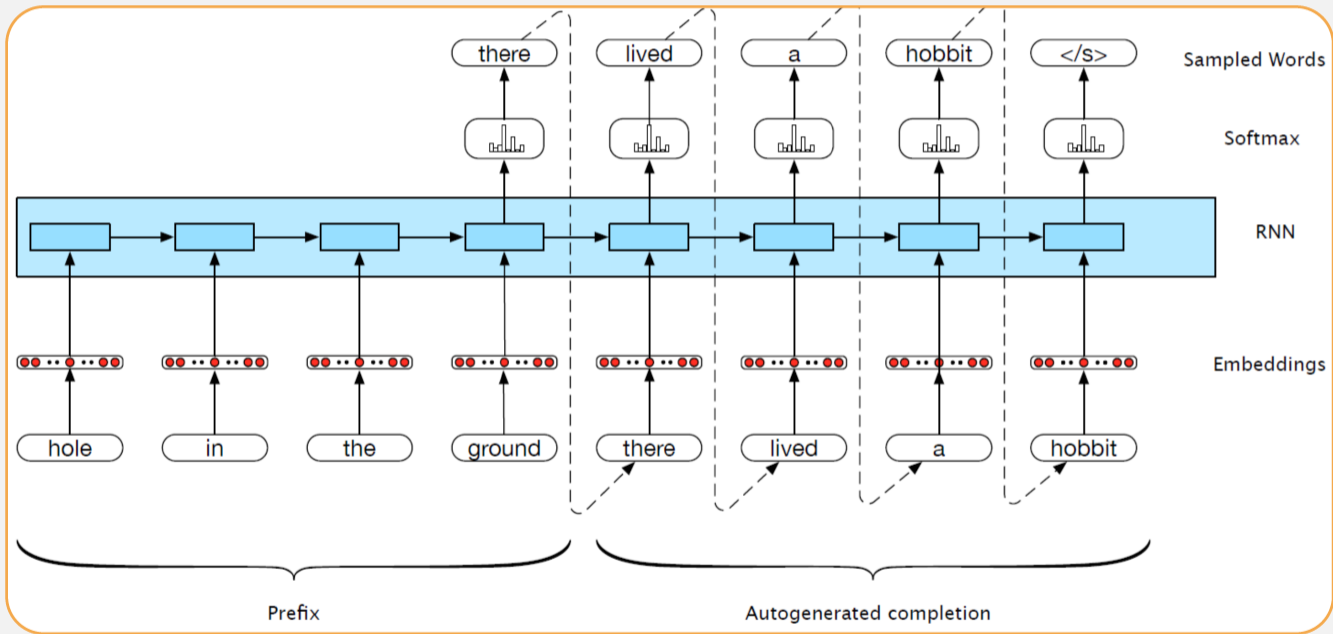


➤ **Encoder-decoder**

➤ **Self Attention Network: Transformers**



Text Generation with RNNs





From (autoregressive) generation to Machine Translation

➤ Training data are parallel text e.g., English / French

▶ there lived a hobbit vivait un hobbit

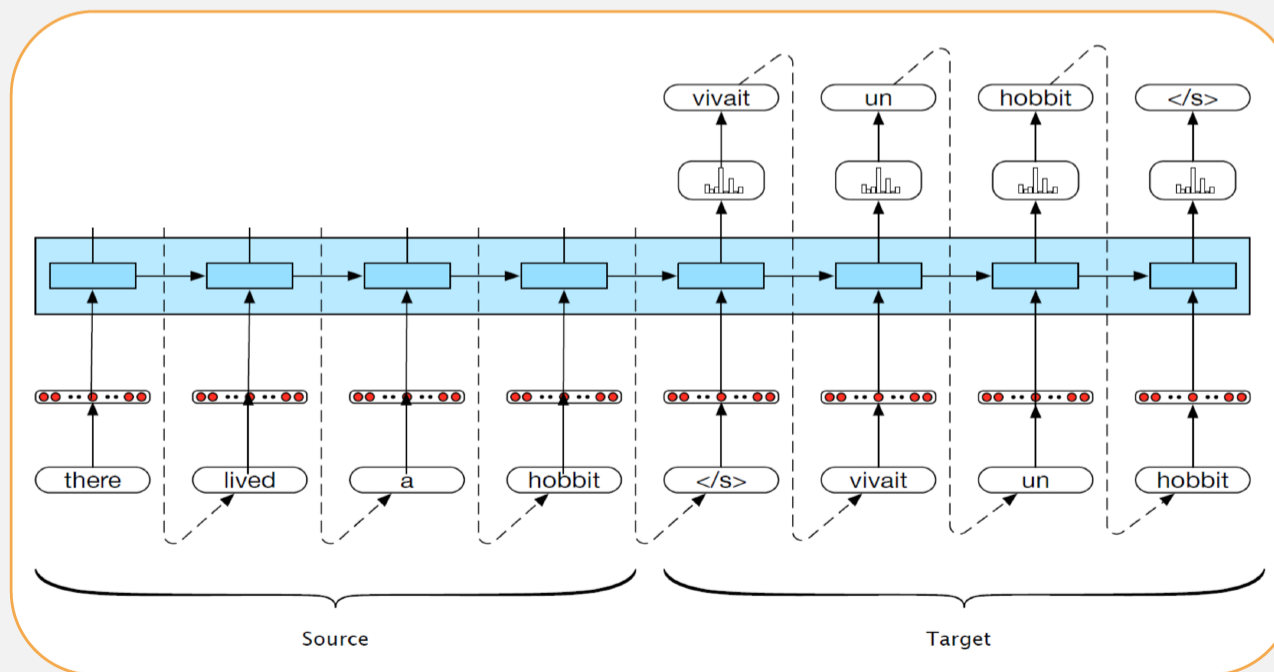
➤ Build an RNN language model on the concatenation of source and target

▶ there lived a hobbit <\s> vivait un hobbit <\s>



From (autoregressive) generation to Machine Translation

➤ Translation as Sentence Completion !





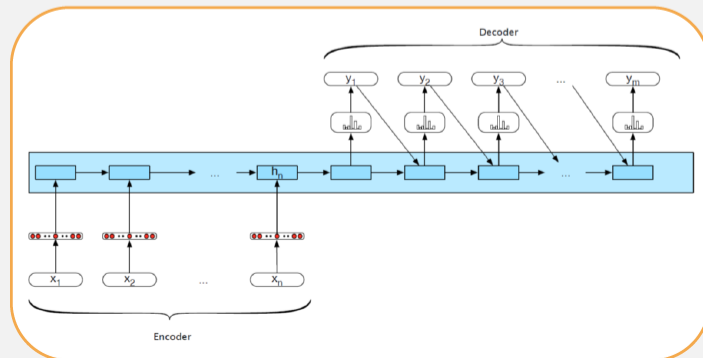
Encoder Decoder Networks

➤ Limiting design choices

- ▶ E and D assumed to have the same internal structure (here RNNs)
- ▶ Final state of the E is the only context available to D
- ▶ This context is only available to D as its initial hidden state

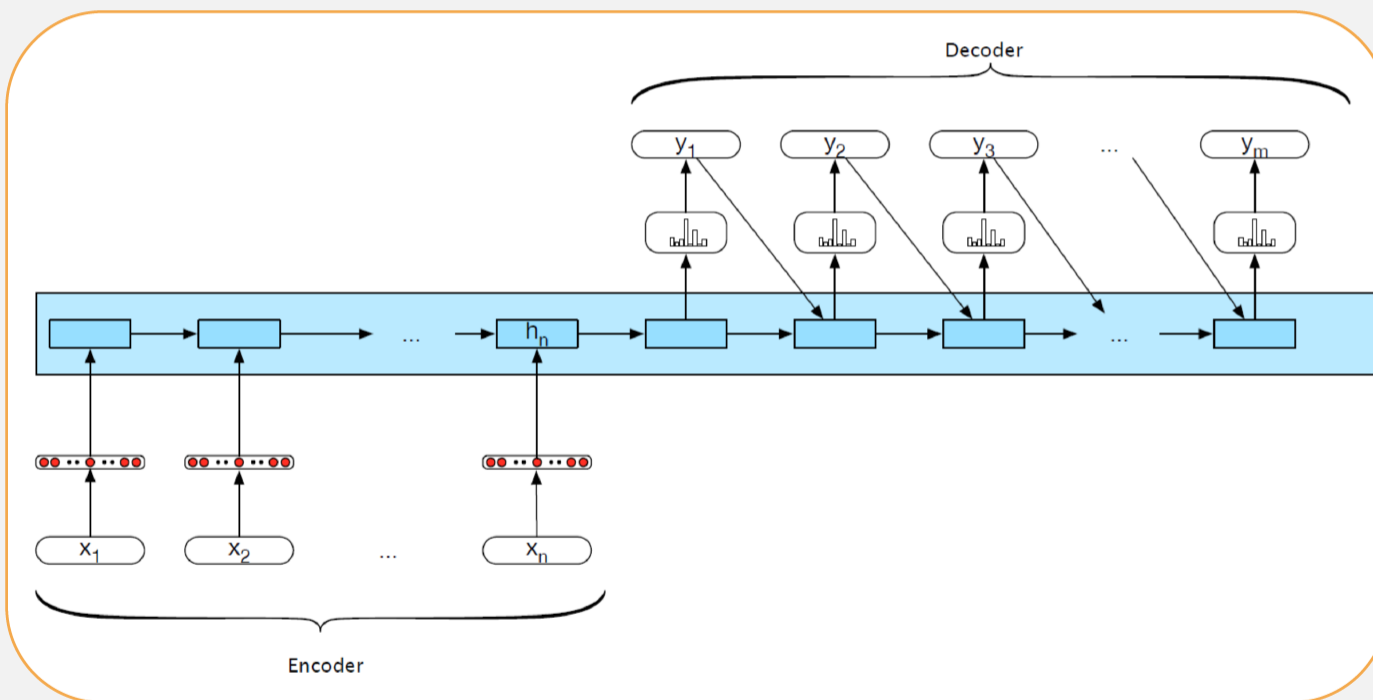
➤ Encoder generates a contextualized representation of the input (last state)

➤ Decoder takes that state and autoregressively generates a sequence of outputs





Encoder Decoder Networks

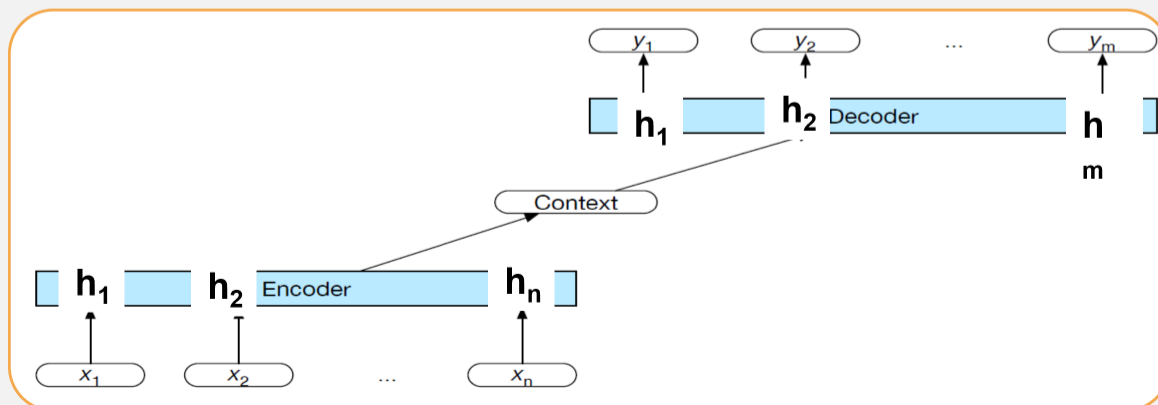




General Encoder Decoder Networks

➤ Abstracting away from these choices

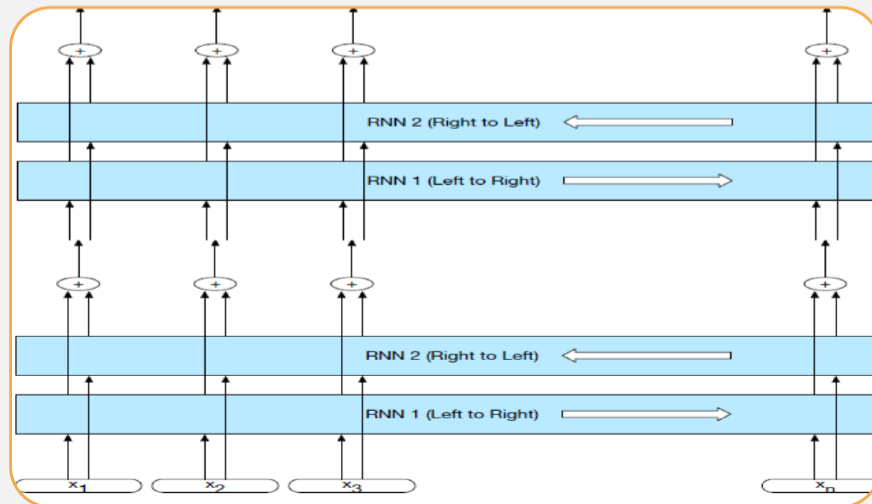
- **Encoder:** accepts an input sequence, $x_{1:n}$ and generates a corresponding sequence of contextualized representations, $h_{1:n}$
- **Context vector c :** function of $h_{1:n}$ and conveys the essence of the input to the decoder
- **Decoder:** accepts c as input and generates an arbitrary length sequence of hidden states $h_{1:m}$ from which a corresponding sequence of output states $y_{1:m}$ can be obtained.





Popular architectural choices: Encoder

- Widely used encoder design: stacked Bi-LSTMs
- Contextualized representations for each time step
 - ▶ Hidden states from top layers from the forward and backward passes



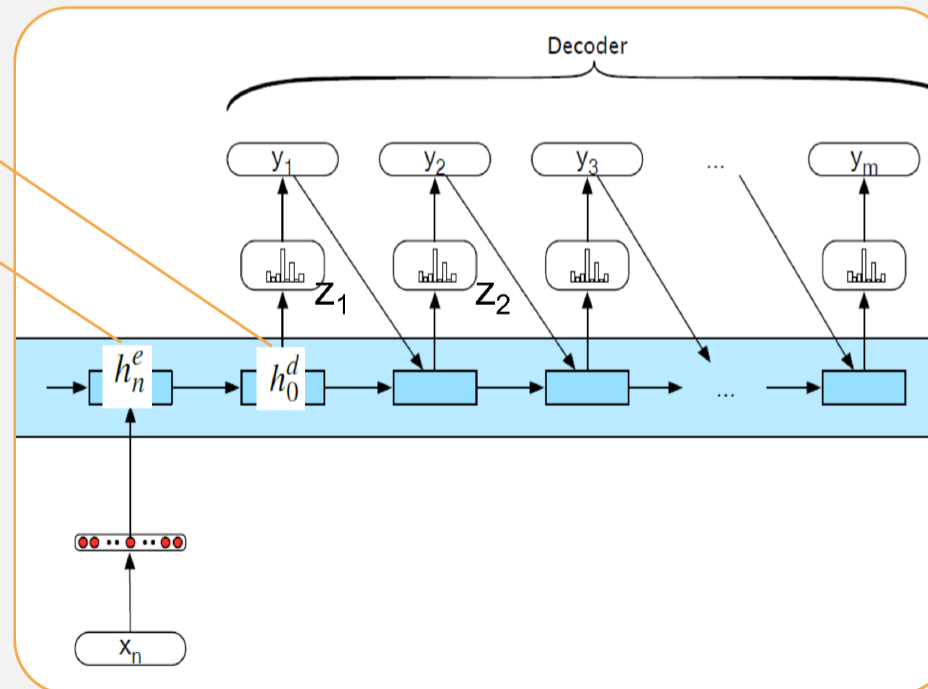


Decoder Basic Design

➤ Produce an output sequence an element at a time

▶ First hidden state of the decoder

▶ Last hidden state of the encoder





» Enhancement

- ▶ Context available at each step of decoding

» How output y is chosen

- ▶ Sample soft-max distribution (OK for generating novel output, not OK for e.g. MT or Summarization)
- ▶ Most likely output (doesn't guarantee individual choices being made make sense together)



Attention Mechanism

➤ Flexible context: Attention

➤ **Context vector c :** function of $h_{1:n}$ and conveys the essence of the input to the decoder.

➤ **Flexible?**

- ▶ Different for each h_i
- ▶ Flexibly combining the h_j



Attention Mechanism

➤ Dynamically derived context

- ▶ Replace static context vector with dynamic c_i
- ▶ Derived from the encoder hidden states at each point i during decoding
- ▶ **Ideas:**
Should be a linear combination of those states

➤ Computing c_i

- ▶ Compute a vector of scores that capture the relevance of each encoder hidden state to the decoder state

➤ Computing c_i From scores to weights

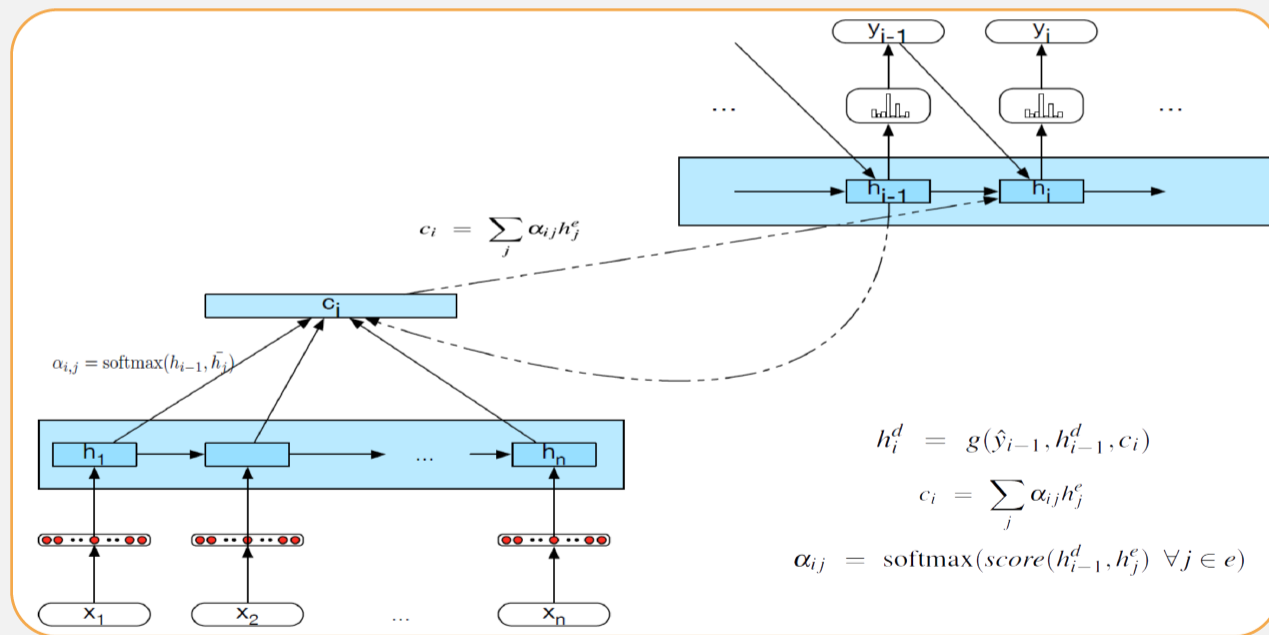
- ▶ Create vector of weights by normalizing scores

➤ **Goal achieved:** compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states.



Attention Mechanism

Encoder



Decoder

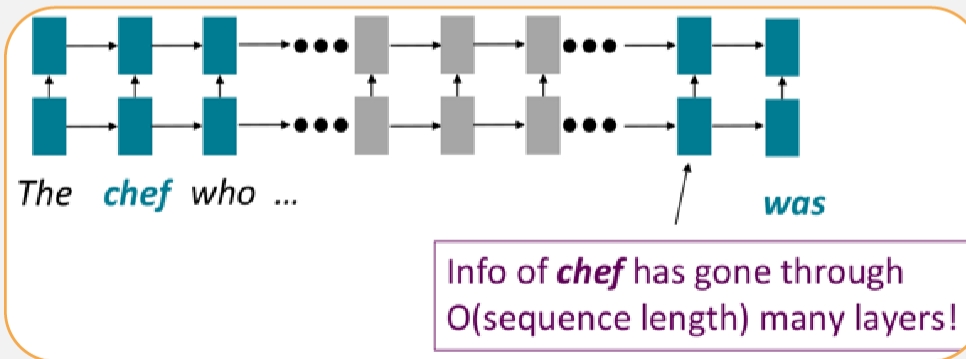


- Encoder-decoder
- **Self Attention Network: Transformers**



RNNs' Shortcomings

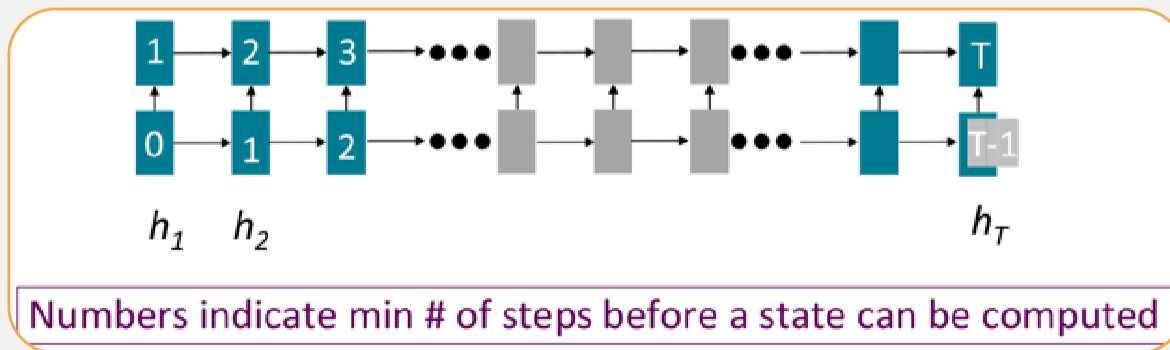
- RNNs take $O(\text{sequence length})$ steps for distant word pairs to interact means
 - ▶ Hard to learn long-distance dependencies (because gradient problems!)
 - ▶ Linear order of words is "baked in"; we already know linear order isn't the right way to think about sentences...





RNNs' Shortcomings

- Forward and backward passes have $O(\text{sequence length})$ unparallelizable operations
 - ▶ GPU can perform a bunch of independent computations at once!
 - ▶ But future RNN hidden states can't be computed in full before past RNN hidden states have been computed
 - ▶ Inhibits training on very large datasets!





If not recurrence, then what? How about word windows?

➤ Word window models aggregate local contexts

- ▶ (Also known as 1D convolution; we'll go over this in depth later!)
- ▶ Number of unparallelizable operations does not increase sequence length!

➤ What about long-distance dependencies?

- ▶ Stacking word window layers allows interaction between farther words

➤ Maximum Interaction distance = sequence length / window size

- ▶ (But if your sequences are too long, you'll just ignore long-distance context)

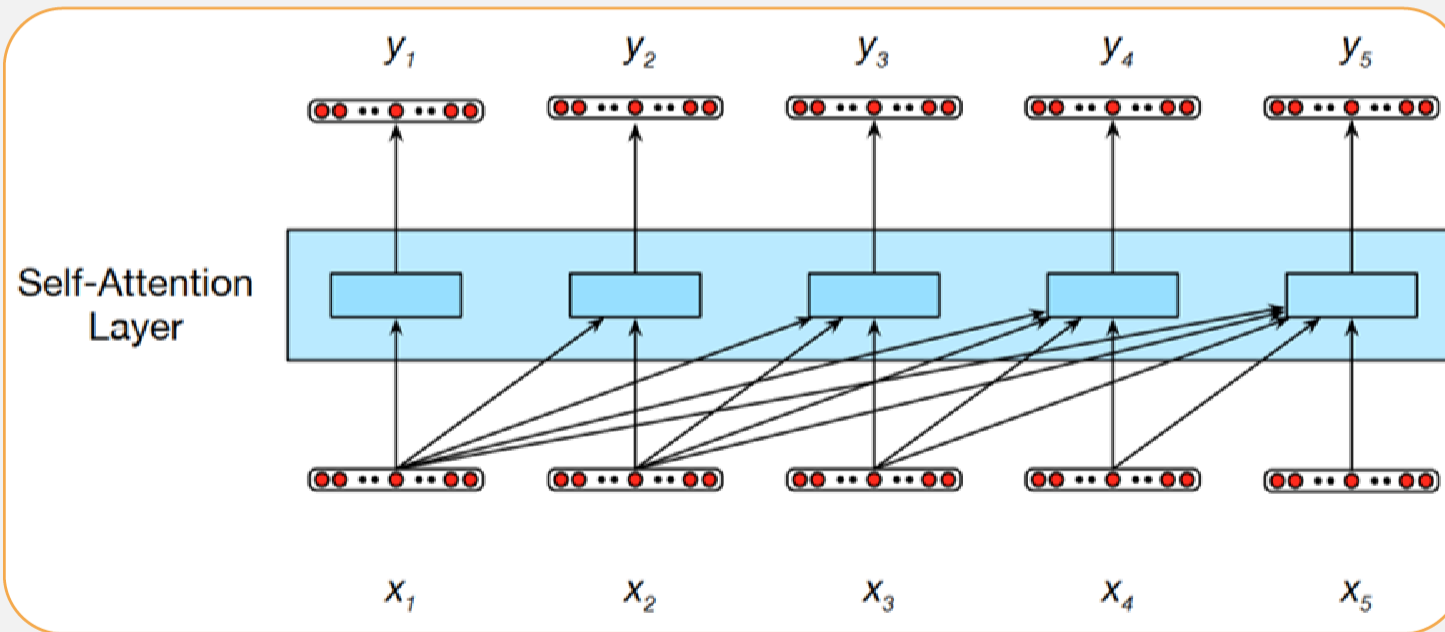


Self Attention

- Attention treats each word's representation as a query to access and incorporate information from a set of values.
 - ▶ We saw attention from the decoder to the encoder;
 - ▶ Now, we'll think about attention within a single sentence.
- Number of unparallelizable operations does not increase sequence length.
- Maximum interaction distance: $O(1)$, since all words interact at every layer!



Self Attention





$$\begin{aligned} \text{score}(x_i, x_j) &= x_i \cdot x_j \\ a_{ij} &= \text{softmax}(\text{score}(x_i, x_j)) \quad \forall j \leq i \\ &= \frac{\exp(\text{score}(x_i, x_j))}{\sum_{k=1}^i \exp(\text{score}(x_i, x_k))} \\ y_i &= \sum_{j \leq i} a_{ij} x_j \end{aligned}$$

- In particular, there are no opportunities to learn the diverse ways that words can contribute to the representation of longer inputs.
- To allow for this kind of learning, Transformers include additional parameters in the form of a set of weight matrices that operate over the input embeddings.



Self Attention

- As the current focus of attention when being compared to all of the other preceding inputs. We'll refer to this role as a query.
- In its role as a preceding input being compared to the current focus of attention. We'll refer to this role as a key.
- And finally, as a value used to compute the output for the current focus of attention.

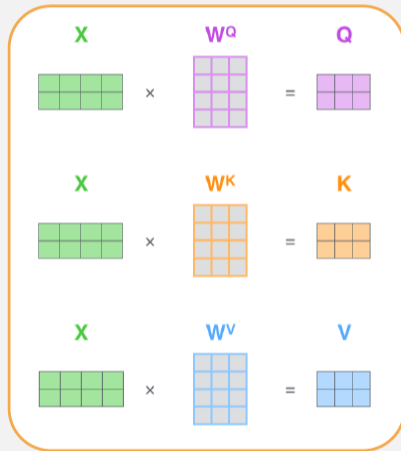
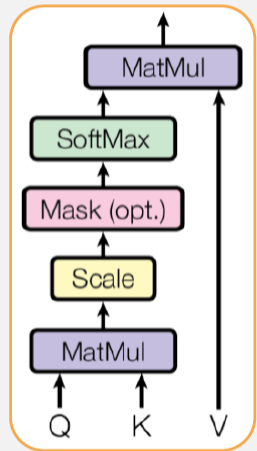
$$score(x_i, x_j) = q_i \cdot k_j$$

$$y_i = \sum_{j \leq i} a_{ij} v_j$$

$$score(x_i, x_j) = (q_i \cdot k_j) / \sqrt{d_k}$$



Self Attention

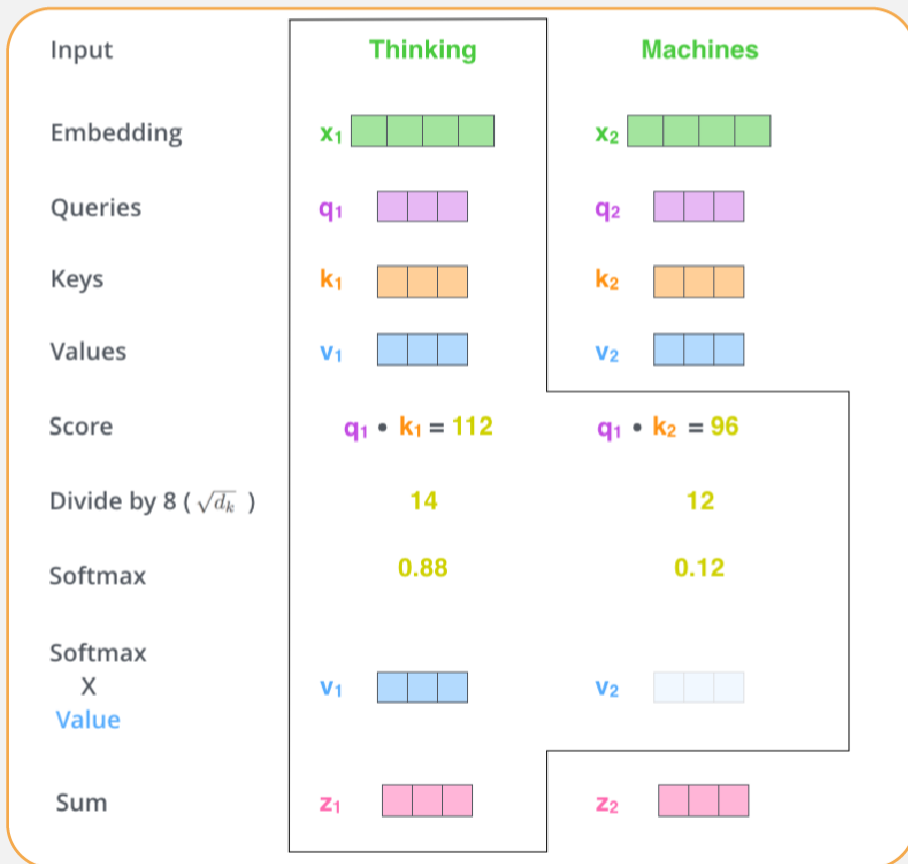


$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

= Z

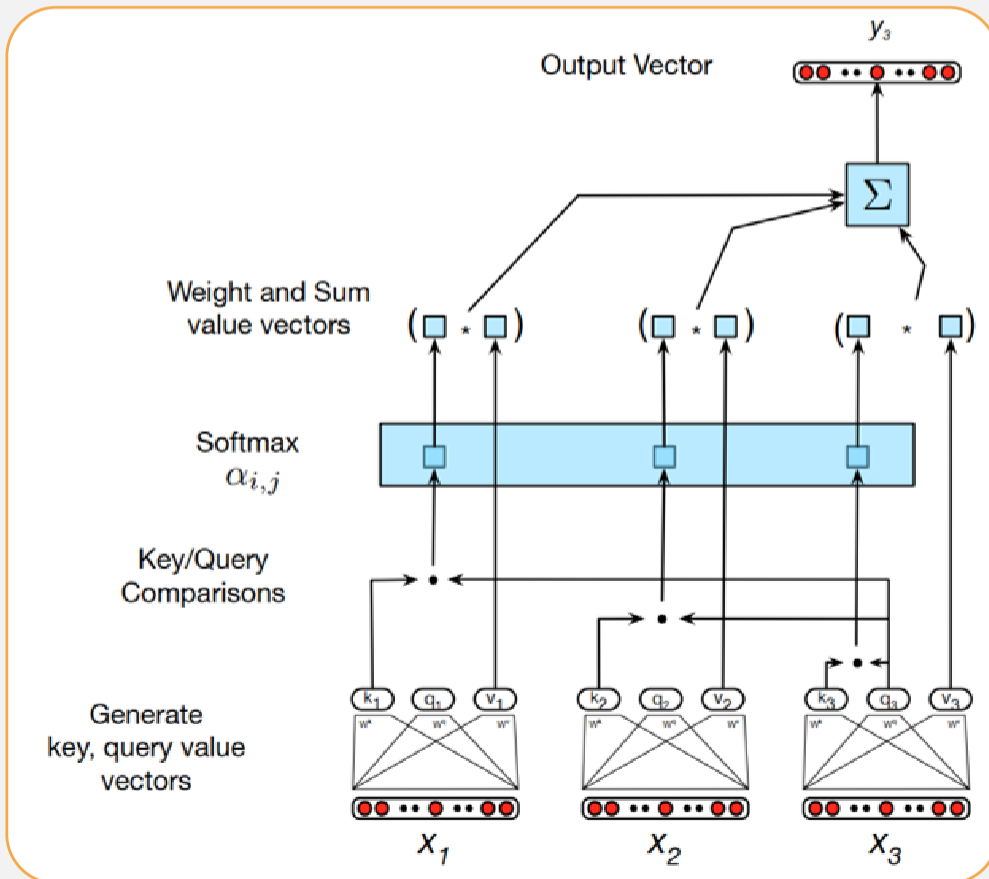


Self Attention





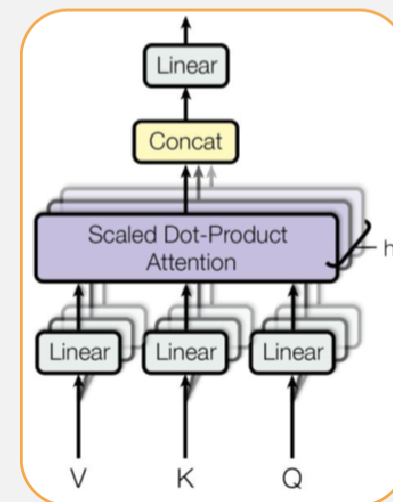
Self Attention





Multihead Attention

- The different words in a sentence can relate to each other in many different ways simultaneously.
 - ▶ Distinct syntactic, semantic, and discourse relationships
- It would be difficult for a single transformer block to learn to capture all of the different kinds of parallel relations among its inputs.
- Transformers address this issue with multihead self-attention layers.
- These are sets of self-attention layers, called heads, that reside in parallel layers at the same depth in a model, each with its own set of parameters.
- Given these distinct sets of parameters, each head can learn different aspects of the relationships that exist among inputs at the same level of abstraction.





Multihead Attention

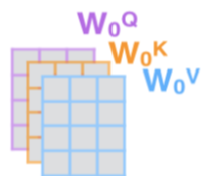
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



3) Split into 8 heads. We multiply X or R with weight matrices



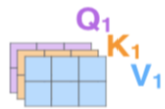
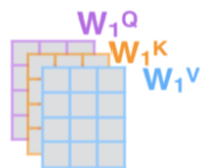
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



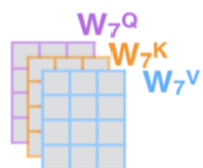
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

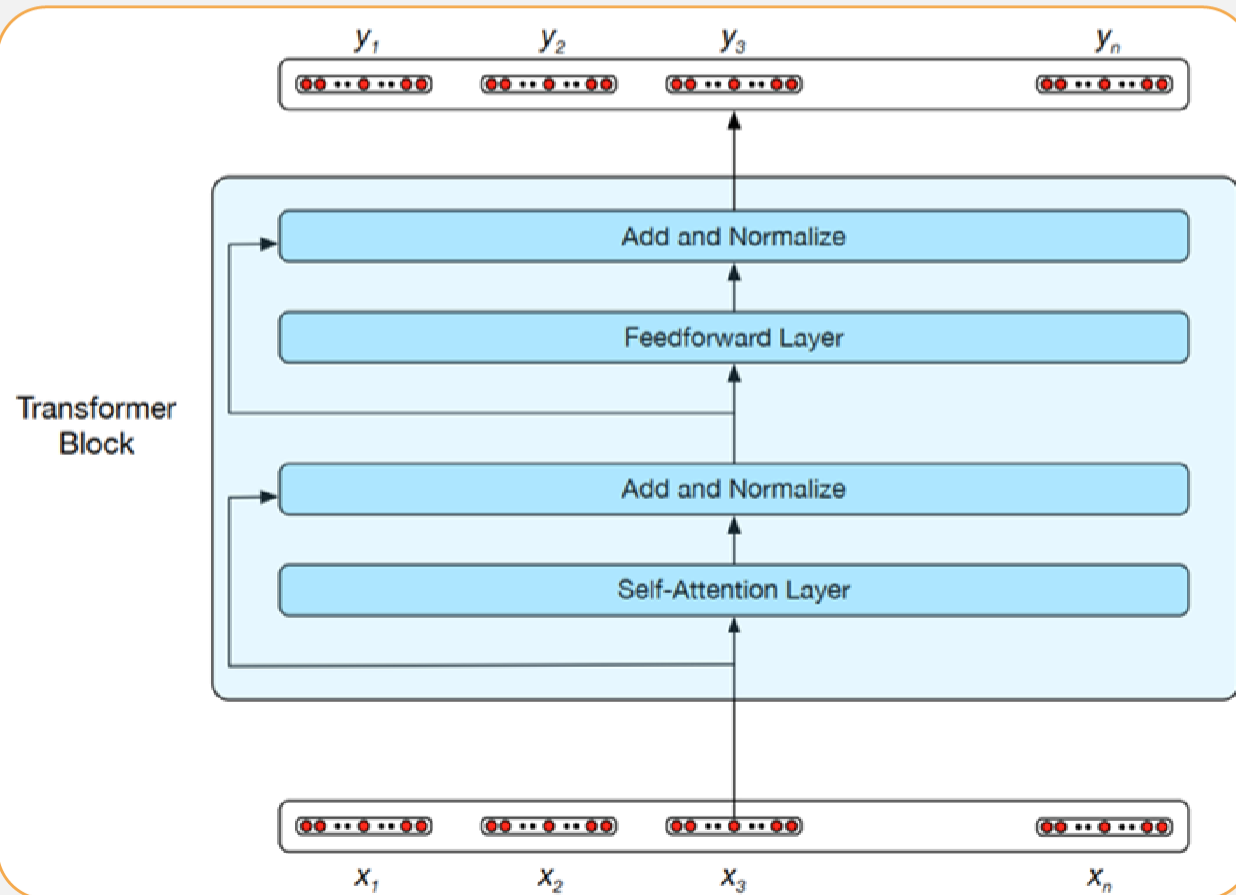
...

...



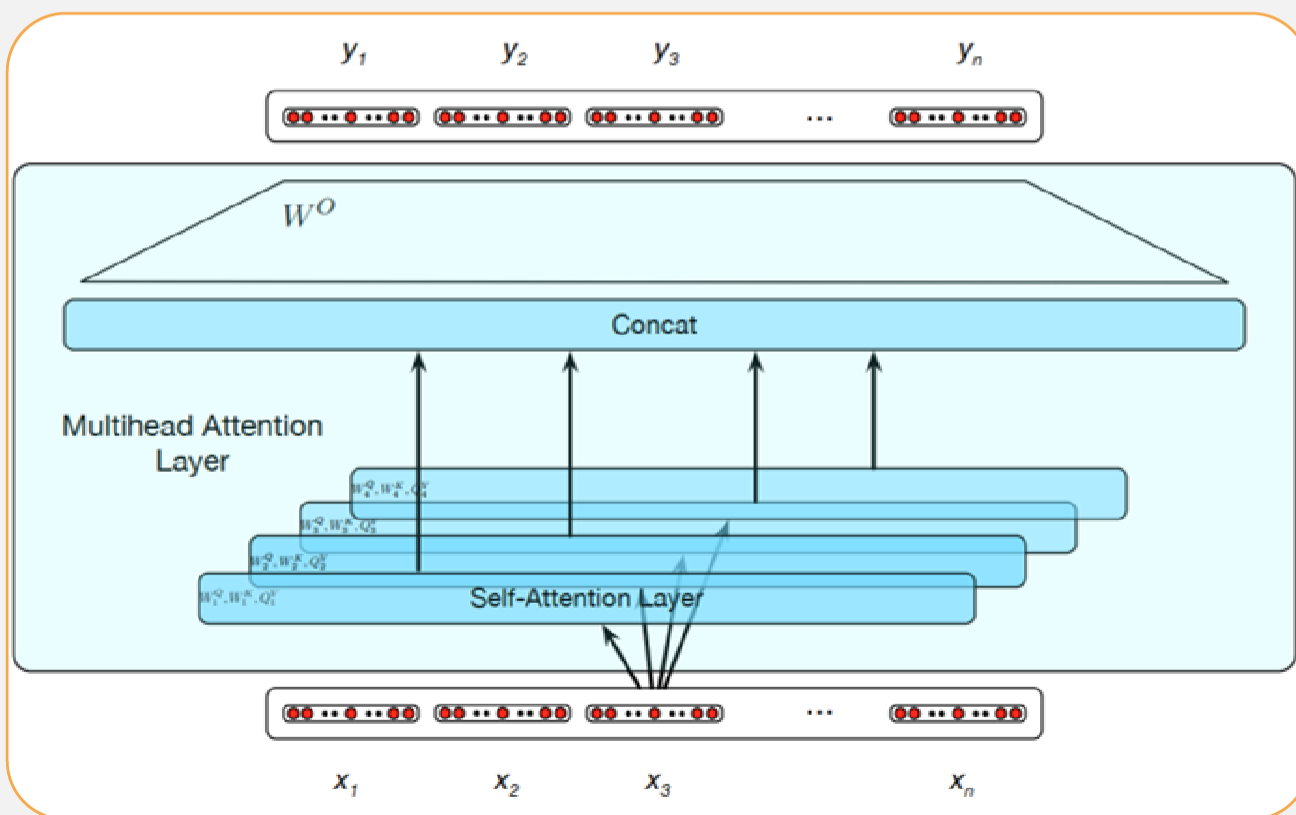


Multihead Attention





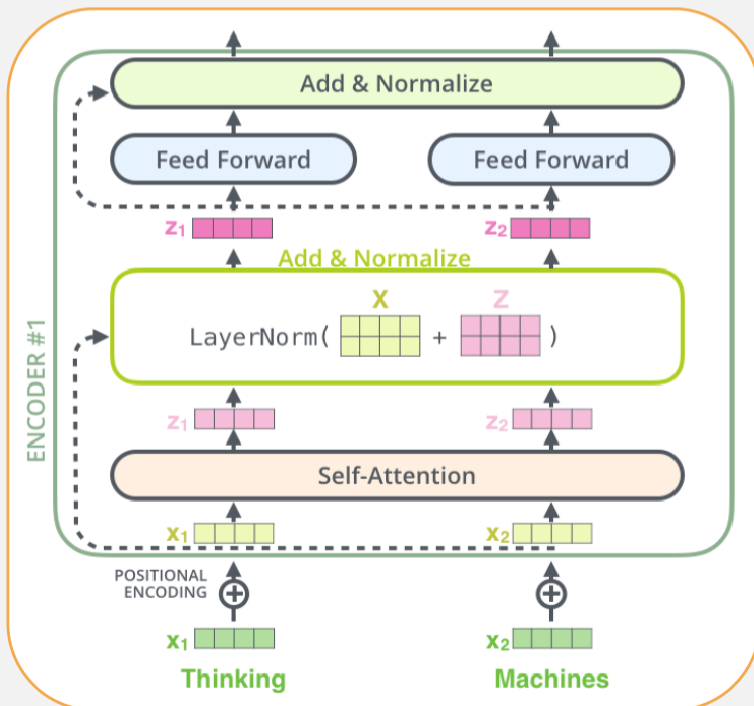
Multihead Attention





Encoder and Decoder in Transformers

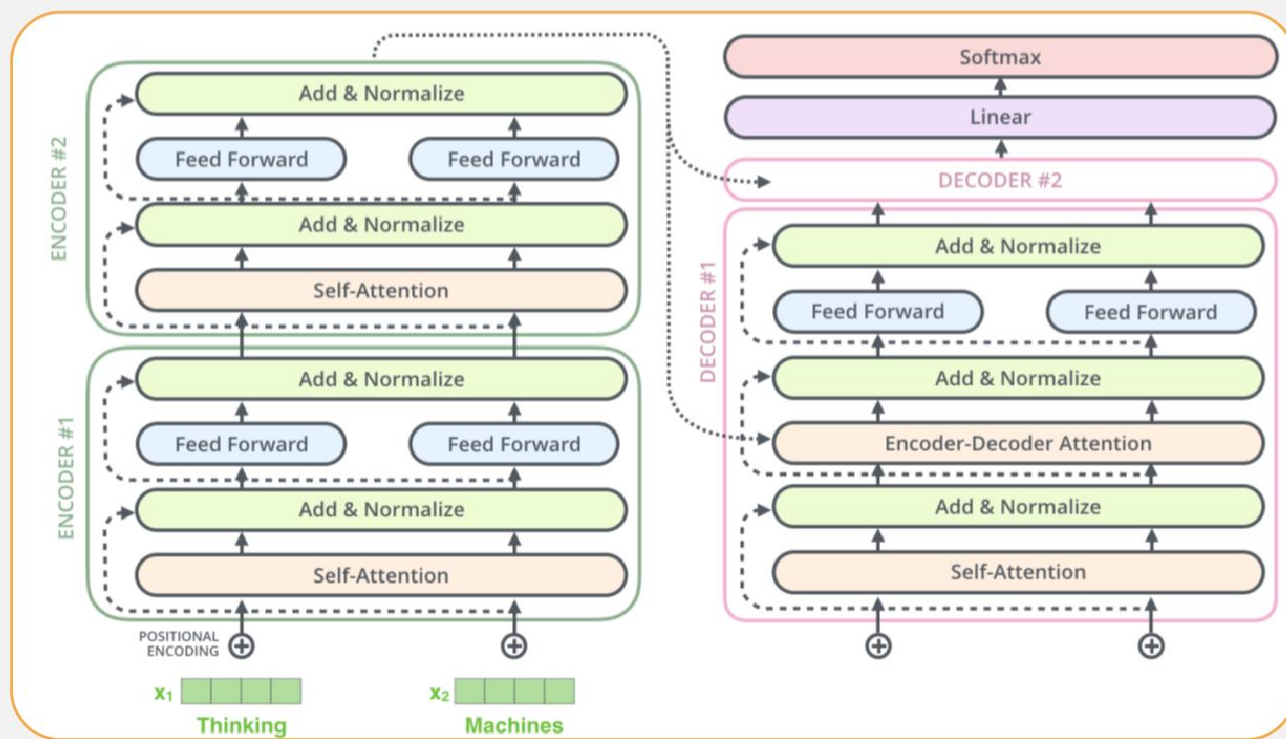
Encoder Structure





Encoder and Decoder in Transformers

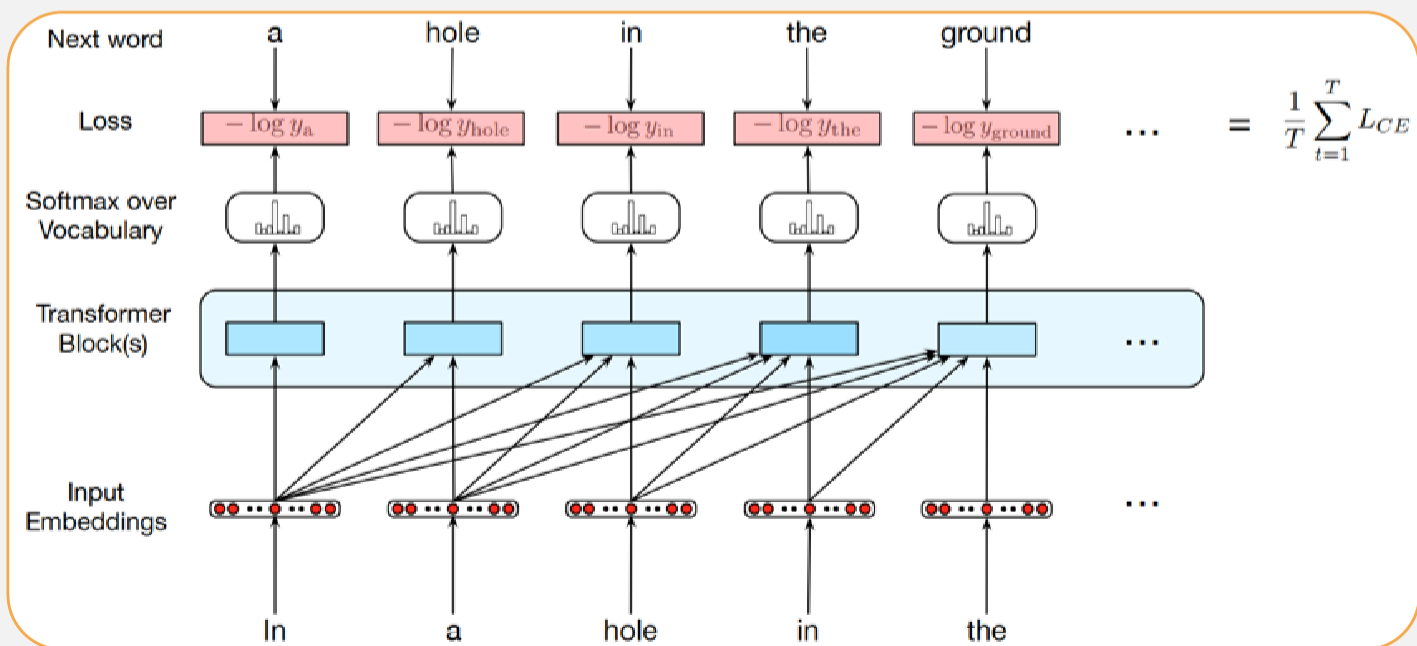
➤ Decoder Structure





Transformers as Autoregressive LMs

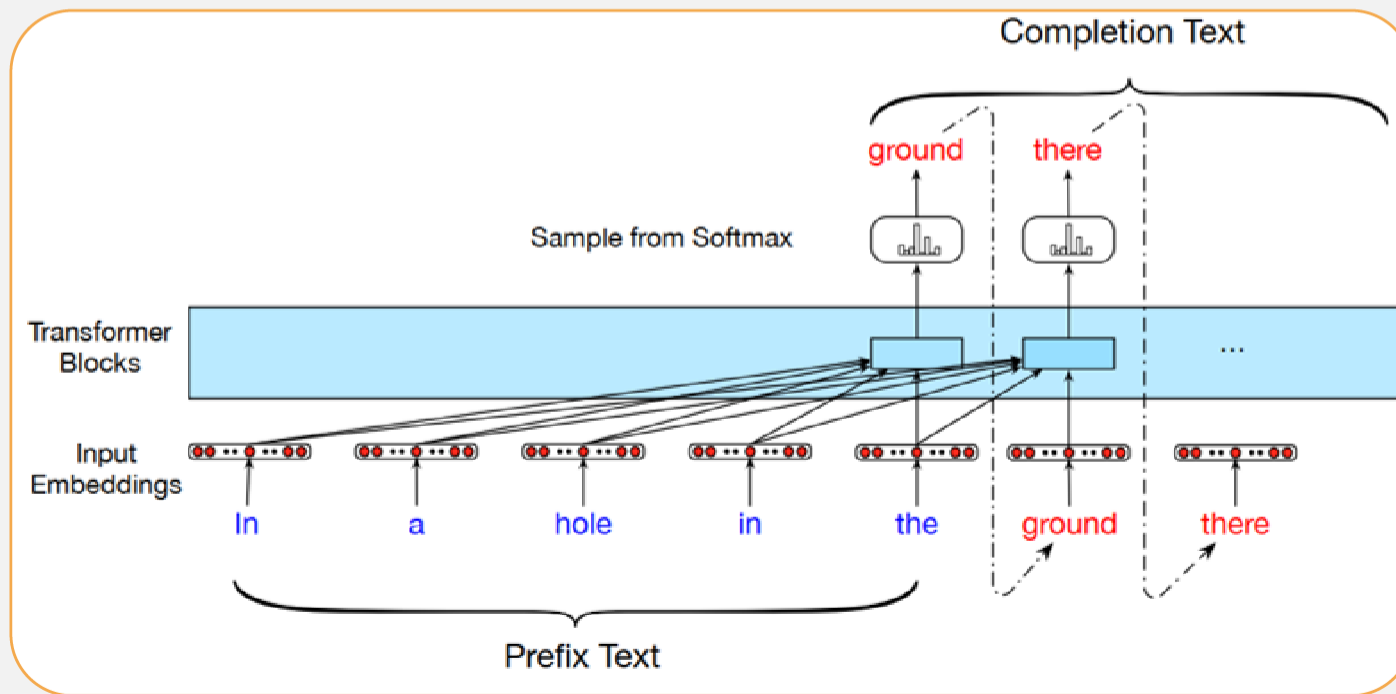
► Training a Transformer as a language model





Transformers as Autoregressive LMs

➤ Autoregressive text completion with Transformers





Other Applications of Transformers

➤ Context Generation

➤ Text Summarization



Text Summarization with Transformers

Original Article

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For \$89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says.

But not if you live in New England or surrounding states. “We will not ship snow to any states in the northeast!” says Waring’s website, ShipSnowYo.com. “We’re in the business of expunging snow!”

His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.

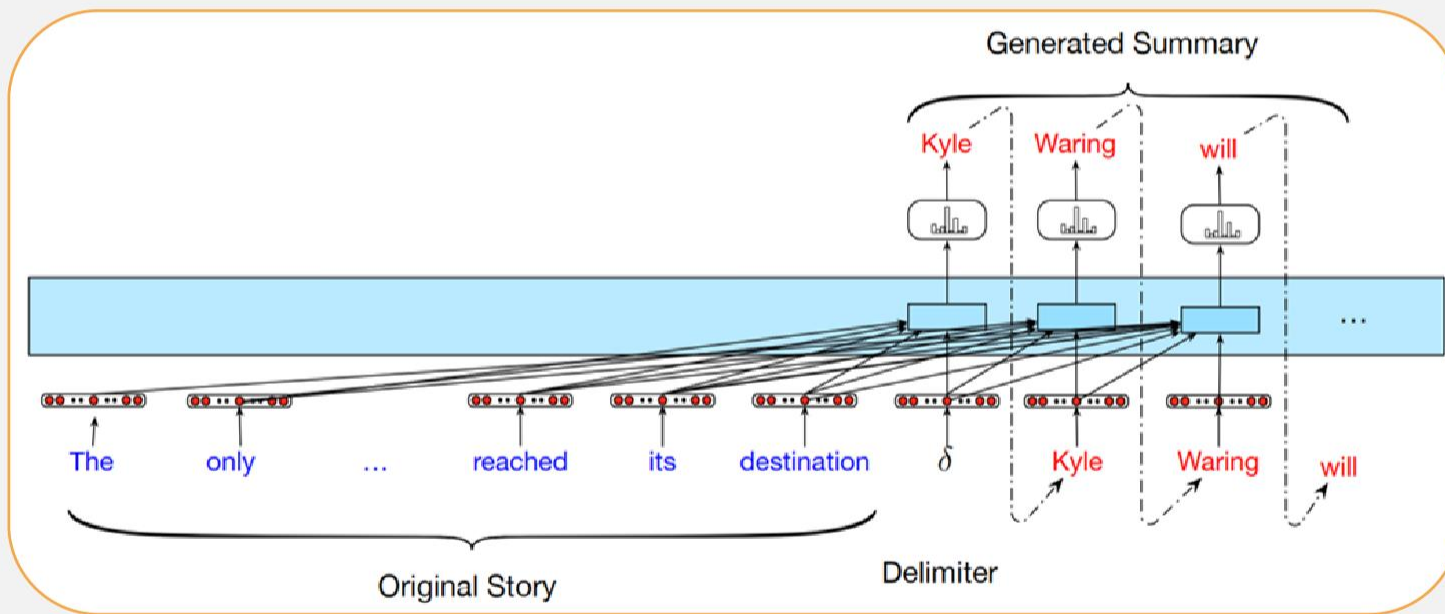
According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. His business slogan: “Our nightmare is your dream!” At first, ShipSnowYo sold snow packed into empty 16.9-ounce water bottles for \$19.99, but the snow usually melted before it reached its destination...

Summary

Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.



Text Summarization with Transformers





Further Reading

➤ **Speech and Language Processing (3rd ed. draft)**

▶ Chapter 9